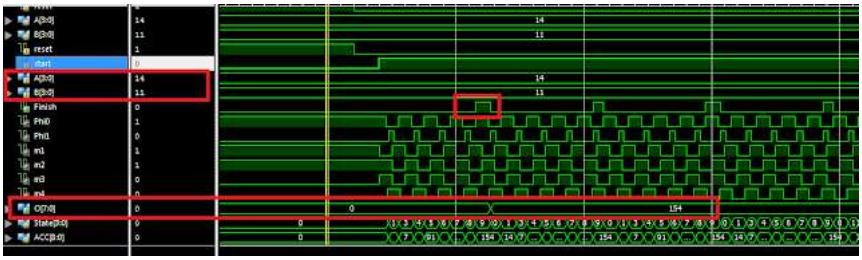
Verilog code for 4x4 Multiplier

This project is to implement a 4x4 multiplier using Verilog HDL. Full Verilog code for the multiplier is presented.
The technique being used is shift/add algorithm, but the different feature is using a two-phase self-clocking system in order to reduce the multiplying time by half.



Verilog code for the multiplier:

```
`timescale 1ns / 1ps
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// multiplier 4x4 using Shift/Add Algorithm and 2-phase clocking system
// Verilog project: Verilog code for multiplier
module mult_4x4(
         input reset,start,
         input[3:0] A,B,
         output [7:0] O, output Finish
             );
reg [7:0] O;
wire Finish;
wire Phi0,Phi1;// 2 phase clocking
wire m1,m2,m3,m4;
// state machine
reg[3:0] State;
// Accumulator
reg [8:0] ACC; // Accumulator
// logic to create 2 phase clocking when starting
nand u0(m1,start,m2);
buf #20 u1(m2,m1);
buf #10 u2(Phi0,m1);// First phase clocking
not #2 u5(m4,Phi0);
assign m3=~m1;
and #2 u4(Phi1,m3,m4);// Second phase clocking
assign Finish = (State==9)? 1'b1:1'b0; // Finish Flag
// FSM
always @(posedge Phi0 or posedge Phi1 or posedge reset)
begin
 if(reset) begin
 State <= 0;
 ACC <= 0;
 O <= 0;
 end
 else if((Phi0==1'b1) || (Phi1==1'b1)) begin // 2 phase clocking
 if(State==0)
 begin
 ACC[8:4] <= 5'b00000; // begin cycle
 ACC[3:0] <= A; // Load A
 State <= 1;
 end
 else if(State==1 || State == 3 || State ==5 || State ==7)
                // add/shift State
 begin
```

```
 if(ACC[0] == 1'b1) begin // add multiplicand
 ACC[8:4] <= {1'b0,ACC[7:4]} + B;
 State <= State + 1;
 end
 else
 begin
 ACC <= {1'b0,ACC[8:1]};// shift right
 State <= State + 2;
 end
 end
 else if(State==2 || State == 4 || State ==6 || State ==8)
                // shift State
 begin
 ACC <= {1'b0,ACC[8:1]}; // shift right
 State <= State + 1;
 end
 else if(State == 9) begin
 State <= 0;
 O <= ACC[7:0];
 end
 end
end

endmodule
// TestBench
// fpga4student.com FPGA projects, Verilog projects, VHDL projects

// Verilog project: Verilog code for multiplier


module test();
// signals
reg start,reset;
reg[3:0] A,B;
// Outputs
wire [7:0] O;
wire Finish;
// device under test
mult_4x4 dut(reset,start, A,B,O,Finish);
initial begin
reset=1; // reset
#40 start = 0;A =14; B= 11;
#400 reset = 0;
#40 start = 1; // start
//@(posedge Finish);
//start = 0;
//$finish;
end

endmodule
```

Simulation result for the 4x4 multiplier:

As soon as the start signal is asserted, the multiplier begins to perform the multiplication. By creating 2 phase clocks, it reduces multiplying time by half.
The finish signal to inform the multiplier that the multiplication has been done and the result is ready.

Simulation result verified the correct operation of the multiplier which 14 multiplies by 11 getting 154 at the output of the multiplier.

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Moore FSM Sequence Detector**
37. **Verilog code for 7-segment display controller on Basys 3 FPGA**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here

Copyright © 2016-2020 FPGA4student.com All Rights Reserved.